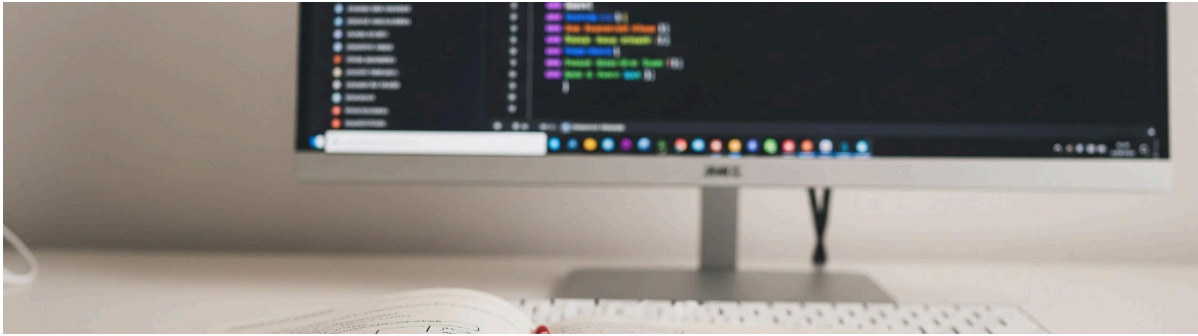




# 개발 일지(Stopwatch, watch)

2026.04.17 - Watch control unit

작업 환경 : Vivado

1) 작업 내용 (What I Did)

| 구분                 | 내용                                                                                                                                                                                                                                                                                                                |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 작업 내용 (What I Did) | WATCH 모듈 제어 로직 최적화 및 조건부 입력 매핑 구현                                                                                                                                                                                                                                                                                 |
| 상세 내역              | <ul style="list-style-type: none"><li>조건부 제어 설계 : <code>if(sw[2])</code> 구문을 활용하여 특정 스위치가 활성화되었을 때만 버튼 입력(<code>i_up</code>, <code>i_down</code> 등)이 출력 신호(<code>o_up</code>, <code>o_down</code> 등)로 전달되는 직관적인 제어 로직 구현</li><li>모듈 간 간섭 제거 : 스위치 상태에 따라 각 버튼의 역할을 명확히 분리하여 시계 설정 모드와 일반 동작 간의 신호 충돌 방지</li></ul> |

2) 회고 및 개선 사항

| 구분   | 내용                                                                                                    |
|------|-------------------------------------------------------------------------------------------------------|
| Keep | 초기 기획했던 복잡한 Moore형 FSM이 오히려 설계 가독성을 해친다는 점을 파악하고, 하드웨어 제어에 더 적합한 '조건부 게이팅' 방식으로 바꿔 구현 속도와 정확도를 높였습니다. |
| Try  | 현재 기획된 다양한 모드(시계, 스톱워치) 간의 전환을 안정적으로 처리하기 위해 가독성이 높은 Moore형 FSM 구조를 도입할 예정입니다. 특히 모드 전환 시 발생할 수       |

| 구분 | 내용                                                         |
|----|------------------------------------------------------------|
|    | 있는 신호 충돌을 방지하기 위해 각 상태별 출력 로직을 명확히 분리하여 설계의 신뢰성을 높여보려 합니다. |

### 3) 이슈 및 해결 과정

| 구분      | 내용                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Trouble | <p>[FSM 구조의 비효율성 및 버튼 로직 꼬임] 모든 입력 시나리오를 FSM의 State로 정의하려다 보니, 상태 전환 조건이 꼬이면서 버튼을 눌러도 값이 변하지 않는 오류가 발생하였습니다.</p>                                                                                                                                                                                                                                                                                                    |
| 해결 과정   | <p>복잡한 상태 머신 대신, 스위치 상태를 직접 체크하는 직관적인 조건문 구조로 코드를 재작성 하였습니다.</p> <p>[수정 코드]</p> <pre> if (sw[2]) begin     if (i_up) begin         o_up  &lt;= 1'b1;     end else if (i_down)         o_down &lt;= 1'b1;     end if (i_right)         o_right &lt;= 1'b1;     else if (i_left) o_left &lt;= 1'b1;     end end </pre> <p>위와 같이 스위치(<b>sw[2]</b>)를 활성화 신호로 활용함으로써, 복잡한 설계 없이도 버튼의 입력과 출력을 안정적으로 연결하고 버그를 해결하였습니다.</p> |

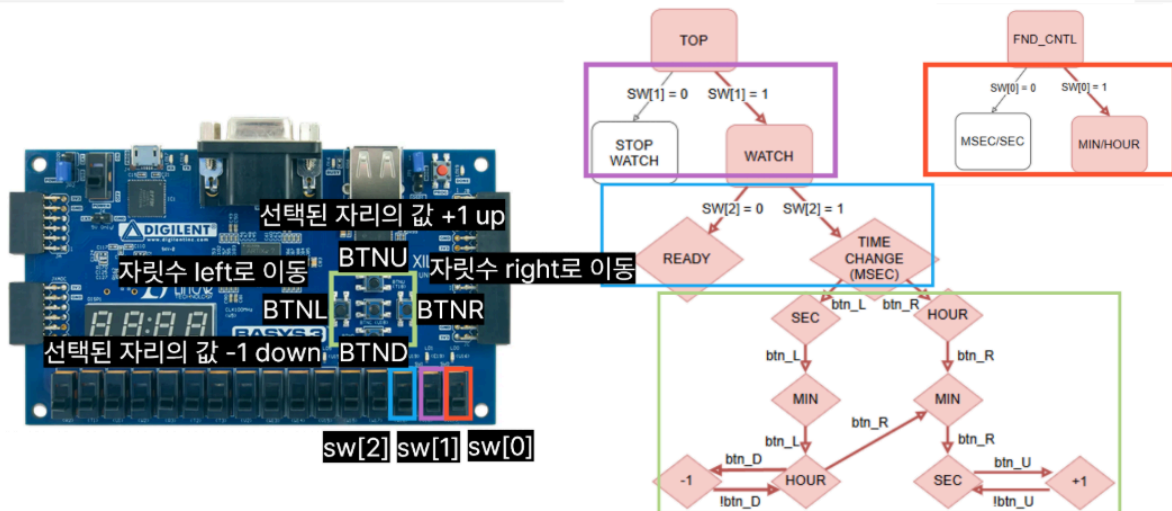
## 3. 팀 토의 필요 사항

- 논의 사항 : 설정 모드 진입 시 시각적 피드백 규격
  - **sw[2]**를 올려 설정 모드에 진입했을 때, 사용자가 현재 '수정 가능 상태'임을 인지할 수 있도록 **FND**를 점멸 시킬지, 혹은 특정 **LED**를 점등시킬지에 대한 **UI/UX** 가이드라인 확정 필요. 디스플레이 제어 로직에 점멸 기능을 추가할 수 있는지 기술적 검토

## 4. 첨부 자료

- 스위치 동작 시나리오

### Watch 구현 기능(버튼, 스위치)



- 코드 스냅샷 :

```
// Safety-gated logic : sw[2] == 1
if (sw[2]) begin
    if (i_up) begin
        o_up <= 1'b1;
    end else if (i_down) begin
        o_down <= 1'b1;
    end else if (i_right) begin
        o_right <= 1'b1;
    end else if (i_left) begin
        o_left <= 1'b1;
    end
end
end
```